

Agentic Development with Harness Engineering

AI-Driven Software Engineering

Humans Steer. Agents Execute.

Ted Jongseok Won

JBUG Korea | April 29, 2026

Disclaimer

이 슬라이드는 AI의 도움을 받아 작성되었습니다.

내용에 오류가 있을 수 있으며, 작성자가 모든 내용을 완벽하게 검증하지 못했습니다.

잘못된 부분이 있다면 알려주시면 감사하겠습니다.

Agenda

00

AI 시대, 개발자의 생존 전략

3 min

01

업무용 AI Assistant — MCP, Agents, Skills

8 min

02

Claude Code 무료 사용 팁

5 min

03

Agentic Development with Harness

22 min

Q&A

핵심 정리 & Q&A

7 min

기술 강의가 아닙니다. 동료 개발자의 실전 경험 공유입니다.

여러분은 지금 AI를 업무에 얼마나 쓰고 계신가요?

채팅으로 답해주세요!

1

안 쓴다

2

가끔

3

매일

"저는 소프트웨어 엔지니어로 **15년+** 일해왔습니다.
그런데 지금 제 업무 방식이 **1년 전과 완전히 달라졌습니다.**"

핵심 메시지

"10년 경력 소프트웨어 엔지니어"만으로는 앞으로 10년을 보장받을 수 없다

AI만으로는 불가능

비즈니스 **맥락** 파악

올바른 **질문** 던지기

최종 **판단**과 **결정**

결과 **검증**과 **책임**

인간만으로는 비효율

방대한 정보 처리

반복 작업

다분야 지식 활용

코드 탐색과 이해

"전문성 alone" vs "전문성 + AI = 압도적 경쟁력"

AI는 전문가를 대체하지 않는다 — 증폭한다

IT는 "대충"이 통하지 않는 분야다 — 정확성이 생명

- AI가 **자신감 있게 틀린 답**을 줄 때가 있다
- 존재하지 않는 API, 잘못된 설정값, 틀린 명령어를 **확신하며** 제시
- AI의 답변을 **100% 신뢰할 수 없다**
- **내가 그 분야의 전문 지식과 경험**이 있어야 AI를 제대로 활용할 수 있다

"AI는 빠르게 달릴 수 있지만, 방향을 아는 건 당신뿐이다."

통제되지 않은 AI는 위험하다

내가 원하는 AI	실제 AI
자세한 내용을 물어보고 진행	알아서 판단해서 진행
한 단계씩 확인	엄청난 양의 결과물을 한꺼번에
내가 컨트롤	내가 압도당함

- 요청 하나에 수십 개 파일을 한 번에 수정
- 물어봤으면 안 했을 작업까지 자기 판단으로 진행

이것이 Harness가 필요한 이유다.

AI의 자율성을 제한하고, 단계별로 검증하는 구조가 필수.

사람이 병목이다

AI 에이전트는 **빠르고 대량으로** 일한다 — 문제는 그 결과를 **사람이 다 확인해야** 한다는 것

- AI가 **5분 만에** 만든 변경사항을 사람이 **1시간** 걸려 리뷰
- 파일 30개를 한 번에 수정 — 사람은 **모든 세부사항을 검토 불가능**
- 리뷰를 포기하면 → **품질 붕괴**, 리뷰를 하면 → **생산성 병목**

그래서 ****자동 검증(Harness)****이 필수다.

사람이 모든 걸 확인하는 대신, **기계가 검증하고 사람은 판단만** 한다.

Part I

업무용 AI Assistant 소개

MCP AI Assistant, Agents, Skills, OpenClaw

MCP AI Assistant — 업무 도구 통합 허브

Julia Evans의 "[Get your work recognized: write a brag document](#)"에서 영감.
AI와 업무 도구를 ****MCP(Model Context Protocol)****로 연결

다양한 서비스, 120+ 도구

CATEGORY	SERVICES
개발	Jira, GitLab, Confluence
커뮤니케이션	Gmail
생산성	Calendar, Drive, Tasks
지식 관리	Obsidian

MCP = AI 모델과 외부 도구 사이의 **표준 인터페이스** (Anthropic 개방형 프로토콜)

MCP AI Assistant 활용 예시

하나의 터미널에서 여러 업무 시스템을 AI가 넘나든다

- > 오늘 내 Jira 티켓 확인해줘
- > 이번 주 캘린더 일정 보여줘
- > Confluence에서 보안 정책 문서 찾아줘
- > PROJ-1234 티켓에 댓글 초안 작성해줘
- > 내일 오전 10시에 보안 리뷰 회의 잡아줘

여러 서비스를 AI 하나로 통합 — 컨텍스트 스위칭 최소화

Claude Code Skills & Agents

한 사람이 33명의 전문가 역량을 활용하는 레버리지 효과

	SKILLS (33개)	AGENTS (4개)
실행	메인 대화에서 inline 실행	독립 context window에서 실행
Software Engineering 비유	<code>import static Utils.*</code>	<code>ExecutorService.submit(task)</code>
비유	매뉴얼 보며 직접 수행	동료에게 위임 후 결과 보고
예시	보안 분석, Prompt 개선, 디버깅	보안 리뷰, 코드 리뷰
속도	빠름	느림 (별도 context)

SKILLS 패키지 매니저

npm처럼 Skill을 설치 / 관리 / 공유

각 분야 전문가 팀을 고용한 것과 같은 효과

OpenClaw — 개인 AI 비서

셀프 호스팅 AI 게이트웨이 (MAC MINI 32GB)

항목	내용
지원 메신저	Discord, Slack, Telegram, WhatsApp, iMessage
라이선스	오픈소스 (MIT)
핵심	내 장비, 내 데이터, 내 규칙

[Telegram]

나: Quarkus health check 어떻게 추가해?

AI: SmallRye Health 확장을 추가하면 됩니다...

"내 주머니 속 AI" — 어디서든 메신저로 AI에 접근

Google Workspace CLI — Google 서비스 AI 연동

Google이 직접 만든 통합 CLI 도구 — 14개 서비스 지원

```
npm install -g @googleworkspace/cli
```

연동 대상	활용 예시
Claude Code	"내일 오전 회의 잡아줘", "이 파일 Drive에 올려줘"
OpenClaw	메신저에서 "오늘 일정 알려줘", "팀에 메일 보내줘"

- 모든 응답이 **구조화된 JSON** → AI가 바로 해석하고 실행
- 40+ Agent Skills 기본 제공

GitHub: github.com/googleworkspace/cli | 참고 영상: youtu.be/S99_UhOQjNw

NVIDIA OpenShell — AI 에이전트 보안 런타임

AI 에이전트를 격리된 샌드박스에서 안전하게 실행

항목	내용
핵심	샌드박스 컨테이너 + YAML 정책으로 접근 제어
지원 에이전트	Claude Code, Codex, Cursor, GitHub Copilot CLI
보안 모델	파일/네트워크/프로세스/추론 4계층 방어
라이선스	Apache 2.0 (오픈소스)

```
openshell sandbox create -- claude    # Claude Code를 샌드박스에서 실행
```

Harness Engineering의 [인프라 수준 구현](#) — 에이전트 프로세스 외부에서 보안 정책 시행

GitHub: github.com/NVIDIA/OpenShell

Part II

Claude Code 무료 사용 팁

Claude Code 시작하기

설치

```
npm install -g @anthropic-ai/claude-code
```

5가지 사용 환경

환경	설명
CLI	터미널에서 <code>claude</code> 실행 — 가장 강력
VS Code Extension	IDE에서 바로 사용
JetBrains Extension	IntelliJ, WebStorm 등
Desktop App	Mac / Windows 전용 앱
Web App	claude.ai/code — 브라우저에서 바로

설치 후 터미널에서 `claude` 입력하면 바로 시작!

무료 / 저비용 사용 방법

방법	비용	특징
Ollama + Claude Code	무료	로컬 모델로 Claude Code UI 사용
Claude.ai 무료 티어	무료	일일 사용량 제한, 기본 체험용
Anthropic API	사용량 기반	가입 시 크레딧, 적게 쓰면 적게 과금
Claude Pro	\$20/월	개인 개발자 추천
Claude Max	\$100/월	헤비 유저, 무제한에 가까운 사용

회사에 AWS/GCP 계정이 있다면 **Bedrock / Vertex AI 경유**로 Claude Code 사용 가능
→ 회사 클라우드 예산으로 과금, 별도 Anthropic 구독 불필요

Claude Code는 어디서 실행되는가?

직접 연결 (기본)

Claude Code CLI → Anthropic API → Anthropic 서버의 Claude 모델

회사 클라우드 경유

Claude Code CLI → Google Vertex AI API → GCP 서버의 Claude 모델

Claude Code CLI → AWS Bedrock API → AWS 서버의 Claude 모델

- **모델은 동일**한 Claude — 실행되는 **서버 위치**만 다름
- Google은 Anthropic의 **주요 투자자** → Vertex AI에 Claude 호스팅
- 회사 GCP/AWS 계약으로 → **추가 벤더 계약 없이** Claude 사용 가능

Ollama로 Claude Code 무료 사용하기

OLLAMA가 CLAUDE CODE의 터미널 UI를 로컬 모델과 연결

```
# Ollama 설치 후 한 줄이면 끝  
ollama launch claude --model kimi-k2.5:cloud
```

항목	내용
동작 방식	Ollama가 Claude Code UI를 로컬/클라우드 모델에 연결
사용 가능 모델	Kimi K2.5, Qwen, DeepSeek, Llama 등
비용	로컬 모델 사용 시 완전 무료
장점	Claude Code의 강력한 UX + 모델 선택의 자유

공식 문서: docs.ollama.com/integrations/claude-code

Claude Code의 UI/워크플로우를 그대로 활용하면서 비용 부담 제로!

CLAUDE.md 메모리 계층

CLAUDE.MD 파일이 여러 곳에 있으면? → 우선순위로 로딩

PRIORITY	LOCATION	SCOPE
1 (highest)	<code>/etc/claude-code/CLAUDE.md</code>	기업/조직 정책 (override 불가)
2	<code>project/CLAUDE.local.md</code>	로컬 오버라이드 (gitignore, 비공개)
3	<code>project/CLAUDE.md</code>	프로젝트 레벨 (팀과 공유, VCS)
4 (lowest)	<code>~/.claude/CLAUDE.md</code>	개인 글로벌 설정 (모든 프로젝트)

- 기업 정책은 **항상 최우선** — 개별 개발자가 override 불가
- 나머지는 **더 구체적인 파일이 우선** (local > project > global)

글로벌에 개인 스타일, 프로젝트에 팀 규칙, 로컬에 내 오버라이드

Andrej Karpathy의 CLAUDE.md 가이드라인

Andrej Karpathy (전 Tesla AI / OpenAI)가 지적한 LLM 코딩의 핵심 문제:

"모델이 당신 대신 잘못된 가정을 하고 그대로 달려간다...
코드를 과도하게 복잡하게 만들고... 이해하지 못하는 코드를 바꾸거나 삭제한다."

4가지 원칙 (CLAUDE.MD로 구현)

원칙	내용
Think Before Coding	가정하지 말고 물어보고 시작
Simplicity First	요청한 것만 구현, 과도한 엔지니어링 금지
Surgical Changes	필요한 부분만 수정, 관련 없는 리팩토링 금지
Goal-Driven Execution	행동을 지시하지 말고 성공 기준을 제공

설치: `/plugin install andrej-karpathy-skills@karpathy-skills`

GitHub: [forrestchang/andrej-karpathy-skills](https://github.com/forrestchang/andrej-karpathy-skills)

settings.json — AI 사고를 사전에 차단

`.claude/settings.json` 에서 AI가 실행할 수 있는 명령어를 **허용/차단** 설정

ALLOW — 허용된 명령어만 실행 가능

```
"allow": [ "Bash(./mvnw *)", "Bash(git status*)", "Bash(git log*)",  
          "Bash(git diff*)", "Bash(git add *)", "Bash(git commit *)" ]
```

DENY — 위험한 명령어 원천 차단

```
"deny": [ "Bash(rm -rf *)", "Bash(git commit --no-verify*)" ]
```

- `rm -rf` → **파일 삭제 차단**
- `--no-verify` → **Harness 우회 차단** (가장 중요!)

settings.json

효율적 사용 팁 4가지

1. CLAUDE.MD 활용

프로젝트 루트에 규칙 파일 → AI가 **자동으로 읽음** → 매번 같은 설명 반복 불필요

2. /COMPACT 로 컨텍스트 압축

대화가 길어지면 **/compact** → 이전 내용 요약 → **토큰 절약**

3. 세션 분리

설계(Design)와 실행(Execute)을 **별도 세션**으로 → 컨텍스트 윈도우 소진 방지

4. SKILLS & PLUGINS 활용

반복 작업을 Skill로 자동화 → 매번 같은 프롬프트 **불필요**

이 4가지만 지켜도 **비용 50% 이상 절약** 가능

Part III

Agentic Development with Harness

github.com/tedwon/agentic-dev-playbook-with-harness

Agentic Development Playbook이란?

AI 에이전트와 사람이 **역할을 나눠** 소프트웨어를 개발하는 체계적 프로세스



Humans steer. 사람이 방향을 잡고 — **Agents execute.** AI가 실행한다.

Agentic AI란? — 학계의 정의

"목표를 달성하기 위해 맥락을 인식하고, 계획하고, 다단계 행동을 자율적으로 실행하는 시스템"

— arxiv.org/pdf/2603.27075

진짜 AI AGENT의 구성 요소

요소	설명
Goal-driven	목표를 받고 자율적으로 실행
Planning	작업을 단계로 분해
Tool Use	API, 코드, 외부 시스템 호출
Memory	단계 간 상태 유지
Loop (ReAct)	Think → Act → Observe → 반복
Multi-agent	여러 에이전트가 협업 (선택)

"Agentic"의 진실 — 솔직하게 말하면

이것은 진짜 AI 에이전트 시스템인가? 아니면 AI를 체계적으로 활용하는 구조인가?

- ✗ 완전 자율 멀티 에이전트 시스템은 아니다
- ○ 에이전트처럼 동작하도록 설계된 구조화된 워크플로우다
- 현실의 대부분 "agentic" 시스템: 하나의 LLM + 순차 호출 + 프롬프트 체이닝

"Agentic은 마케팅 용어로 자주 사용된다" — arxiv.org/pdf/2506.01463

진짜 에이전트가 서로 대화하는 것이 아니라, 오케스트레이션된 LLM 워크플로우

Agentic 성숙도 — 우리는 어디에 있는가?

LEVEL	설명	예시
0	단일 LLM 응답	ChatGPT에 질문 하나
1	프롬프트 엔지니어링	체계적인 프롬프트 설계
2~3	구조화된 워크플로우 + 도구 사용	← 우리 Playbook
4	진짜 멀티 에이전트 시스템	AutoGen, LangGraph, CrewAI

Agentic ≠ 여러 AI가 자율적으로 협업

현실: Agentic = LLM + Loop + 구조화된 프롬프트 + 도구 사용

그래도 이것만으로 실무에서 충분한 가치를 만들어내고 있다.

Playbook의 장점

구조 없이 AI에게 "만들어줘"라고 하면?

- AI가 **방향 없이** 코드를 쏟아냄 → 원하지 않는 결과물
- "알아서 판단하는 AI" 문제 그대로 발생

Playbook이 해결하는 것

- **설계 → 실행 분리**: 사람이 먼저 방향을 잡음
- **Human-in-the-loop**: 각 단계마다 사람이 확인하고 승인
- **Spec + Plan 문서화**: AI가 뭘 만들지 미리 합의 (새 세션으로 핸드오프 가능)

Playbook 덕분에 AI가 **엉뚱한 방향**으로 달려가는 건 막을 수 있게 되었다.

그런데 Playbook만으로는 부족했다

- AI가 **컴파일 안 되는 코드**를 자신있게 커밋
- **System.out.println**, 하드코딩된 비밀번호 같은 기본적 실수
- 테스트 파일 **없이** 기능만 구현
- 심지어 **규칙 파일 자체를 수정**해서 검증을 우회하려는 시도

Playbook은 "**무엇을 할지**" 를 잡아주지만
"**제대로 했는지**" 를 자동으로 검증하는 구조가 없었다.

그래서 Harness Engineering을 도입했다

Playbook + Harness = 만족스러운 수준

	PLAYBOOK만	PLAYBOOK + HARNESS
방향 설정	O	O
자동 검증	X	7가지 자동 검증
실수 자동 교정	X	Self - Correction Loop
규칙 파일 보호	X	File Protection Hook
즉시 피드백	X	Post-edit 컴파일 확인

Playbook이 **방향타**라면, Harness는 **가드레일**이다.
둘을 합쳐서 비로소 AI 에이전트를 **믿고 맡길 수 있는** 수준이 되었다.

Harness Engineering의 탄생

2026년 2~3월, "HARNESS"라는 용어가 업계에 등장하고 확산되다

날짜	저자	핵심 기여
2026. 2. 5	Mitchell Hashimoto	"Engineer the Harness" 개념 최초 제안
2026. 2. 11	OpenAI (Codex 팀)	<code>Agent = Model + Harness</code> 공식화
2026. 3. 24	Anthropic	Harness Design 심화 — 3-Agent 아키텍처
2026. 4	이 프로젝트	Harness Engineering 실전 적용 — JBUG Seminar

3개월 만에 개인 블로그의 아이디어가 → OpenAI/Anthropic의 공식 엔지니어링 방법론으로 확산
그리고 우리는 이것을 **Quarkus 프로젝트에 직접 적용**했다

Mitchell Hashimoto — "Engineer the Harness" (2026. 2. 5)

HashiCorp 창업자가 자신의 **AI 도입 6단계**를 공개

PHASE	단계	내용
1	Drop the Chatbot	챗봇을 넘어 에이전트 사용
2	Reproduce Your Own Work	같은 작업을 수동 + AI로 반복하며 학습
3	End-of-Day Agents	퇴근 시 에이전트에게 리서치 위임
4	Outsource the Slam Dunks	확실한 작업을 에이전트에게 위임
5	Engineer the Harness	에이전트 실수를 구조적으로 방지
6	Always Have an Agent Running	항상 에이전트가 돌아가는 상태

"에이전트가 실수할 때마다, 그 실수가 다시는 발생하지 않도록 구조를 만든다."

- **AGENTS.md**: **"그 파일의 모든 줄은 에이전트의 나쁜 행동에 기반한 것이다"**

원문: mitchellh.com/writing/my-ai-adoption-journey

OpenAI — Agent = Model + Harness (2026. 2. 11)

- 소수의 엔지니어(3→7명), 코드 0줄, PR 1,500개, 100만 줄 — 모두 에이전트가 작성
- Agent = Model + Harness 공식 제시
- LangChain: Harness만 개선 → Terminal Bench 2.0 30위 → 5위 (모델 변경 없이)

"어려운 건 Agent가 아니라 Harness다." — OpenAI

ANTHROPIC — HARNESS DESIGN 심화 (2026. 3. 24)

- 다중 에이전트 하네스: Initializer(작업 분해) → Coding Agent(구현) → 검증 루프
- Solo agent (\$9, 동작 안 함) vs Harness (\$200, 완벽 동작) → 비용이 아니라 구조가 품질을 결정

"하네스의 모든 컴포넌트는 모델이 혼자서는 못하는 것에 대한 가정을 인코딩한다."

OpenAI: openai.com/index/harness-engineering · Anthropic: anthropic.com/.../harness-design-long-running-apps

우리 프로젝트에 적용한 것

원천	우리의 구현
Hashimoto의 AGENTS.md	CLAUDE.md + AGENTS.md + CHECKLIST.md *
OpenAI의 Feedforward/Feedback	Pre-commit Hook (7 checks) + File Protection
Anthropic의 세션 분리	Phase 1(설계)과 Phase 2(실행) 별도 세션
Anthropic의 Evaluator	Reviewer 서브에이전트 + Human Review

* CHECKLIST.md 는 외부 참고 자료에 없는 **우리의 독자적 아이디어** — 검증 규칙을 선언적으로 문서화

블로그 글의 **이론**을 → 실전 프로젝트의 **코드**로 구현

GitHub: github.com/tedwon/agentic-dev-playbook-with-harness

AI 코딩의 현실

개발자의 **80%** 가 AI 코딩 도구를 사용하고 있지만... — Stack Overflow 2025

- AI가 생성한 코드의 **40%에 보안 취약점** 존재 — Pearce et al. 2022
- 프로덕션에 AI 코드를 **신뢰**하는 개발자는 **29%에 불과** — Stack Overflow 2025
- AI가 만든 코드를 리뷰 없이 머지한 팀에서 **장애 발생 사례** 증가

근본 원인

AI 모델의 능력이 부족한 것이 아니다.
검증 구조(Harness)의 부재가 문제다.

Agent = Model + Harness

	MODEL (엔진)	HARNESS (제어 장치)
역할	코드 생성, 추론, 분석	검증, 교정, 가드레일
비유	700마력 엔진	핸들 + 브레이크 + 계기판
단독	빠르지만 위험	느리지만 안전
결합	빠르고 안전	

Harness 설계가 모델 선택만큼 중요하다.

좋은 모델 + 나쁜 Harness = 위험한 속도.

보통 모델 + 좋은 Harness = 안정적인 성과.

Harness Engineering이란?

AI 에이전트가 **자율적으로 일하되, 안전하게** 일할 수 있는 구조를 설계하는 것

두 가지 제어 방식

제어 유형	언제?	어떻게?	구현 파일
Feedforward (예방)	실수 전에	규칙을 미리 읽게 함	CLAUDE.md , AGENTS.md
Feedback (교정)	실수 후에	에러 감지 → 수정 유도	Pre-commit hooks

핵심 원리:

에이전트가 규칙을 읽고(Feedforward), 위반 시 스스로 고치고(Feedback),
사람 개입 없이 반복한다.

Self-Correction Loop

사람의 개입 없이 에이전트가 스스로 교정하는 핵심 메커니즘

```
Agent가 코드 작성
|
git commit 시도
|
Pre-commit Harness → 7가지 검증 실행
|-- 모두 통과? --> Commit 성공
|-- 실패? --> Commit 차단 + 상세 에러 메시지
|
Agent가 에러를 읽고 수정
|
다시 commit 시도 (반복)
```

이 루프가 작동하려면 에러 메시지가 **LLM이 이해할 수 있는 형태**여야 한다.

LLM-Optimized 에러 메시지

Harness의 에러 메시지는 사람이 아닌 **AI 에이전트를 위해** 설계됨

```
[FAIL] QUAL-01 | System.out.println found in QuoteResource.java
```

```
WHAT: System.out.println("Loading quotes...");
```

```
WHY: Production code must use structured logging.
```

```
FIX: Replace with org.jboss.logging.Logger
```

```
EXAMPLE:
```

```
private static final Logger LOG = Logger.getLogger(QuoteResource.class);  
LOG.info("Loading quotes...");
```

에러 메시지 구성 요소

Rule ID → 무엇이 틀렸는지 → 왜 틀렸는지 → 어떻게 고치는지 → 코드 예시

7가지 자동 검증 규칙

모든 `git commit` 전에 **7개 검증을 동시에** 실행

ID	CHECK	내용
BUILD-01	컴파일 검증	<code>./mvnw compile -q</code>
BUILD-02	테스트 검증	<code>./mvnw test</code>
BUILD-03	포매팅 검증	<code>./mvnw spotless:check -q</code>
QUAL-01	System.out 금지	<code>org.jboss.logging.Logger</code> 사용 강제
QUAL-02	시크릿 금지	<code>@ConfigProperty</code> 사용 강제
CONV-01	Conventional Commits	<code>feat(scope): subject</code> 형식
CONV-02	테스트 커버리지	모든 <code>@Path</code> 클래스에 <code>*Test.java</code> 필수

설계 원칙: 모든 실패를 **한 번에** 보여줘야 에이전트가 **한 번에** 고친다.

3가지 Hook 시스템

HOOK	시점	동작	실패 시
Pre-commit Harness	<code>git commit</code> 시	7가지 검증 실행	Commit 차단
File Protection	파일 수정 시	규칙 파일 변조 감지	수정 차단
Post-edit Verify	<code>.java</code> 편집 후	즉시 컴파일 확인	경고만

File Protection이 필요한 이유:

에이전트가 검증 실패를 "해결"하는 **가장 쉬운 방법**은
규칙 자체를 수정하는 것이다. 이를 **원천 차단**한다.

데모 프로젝트: Quote of the Day API

이 프로젝트 전체가 **Agentic Development Playbook**으로 개발되었다.

REST API ENDPOINTS

GET /api/quotes	전체 명언 목록 (?category=programming 필터)
GET /api/quotes/random	랜덤 명언 하나
GET /api/quotes/{id}	ID로 조회 (없으면 404)

TECH STACK

Java 21 + Quarkus 3.34 + Spotless + REST Assured + JUnit 5 (11개 테스트)

GITHUB

github.com/tedwon/agentic-dev-playbook-with-harness

Playbook, Hook 스크립트, 데모 녹화 모두 공개

데모: 위반 코드 작성

의도적으로 **4가지** 규칙을 위반하는 코드를 작성

```
public class TimeResource {  
    @GET  
    public String getTime() {  
        System.out.println("time requested");    // QUAL-01 위반  
        return LocalDateTime.now().toString();    // BUILD-03 위반 (들여쓰기)  
    }  
}  
  
// 테스트 파일 없음    // CONV-02 위반  
// git commit -m "added stuff"    // CONV-01 위반
```

데모: Harness가 차단하고 교정한다

```
[FAIL] QUAL-01 : System.out.println found → Use Logger  
[FAIL] BUILD-03 : Formatting violation → Run spotless:apply  
[FAIL] CONV-02 : No test for TimeResource → Create TimeResourceTest.java  
[FAIL] CONV-01 : Bad commit msg → Use feat(quotes): add time endpoint
```

```
RESULT: 3/7 passed, 4/7 FAILED - commit blocked
```

AGENT의 자동 교정 (사람 개입 없음)

Agent가 4개 위반을 **한 번에** 읽고, **한 번에** 수정 → 재시도 → **7/7 통과**

실전 데모: Issue #2 — 명언 AI 챗봇

Agentic Development Playbook 4단계를 실제로 적용하여 AI 챗봇 기능을 구현

PHASE	내용	산출물
Design	브레인스토밍 → Spec → Plan	Spec · Plan
Execute	7 Tasks 구현 → Harness 7/7 통과	새 파일 6개 + 수정 3개
Review	PR 생성 → 코드 리뷰	PR #3
Validate	SpotBugs + SBOM + 로컬 테스트	13 tests passed (base 11 + 신규 2)

Issue: #2 · PR: #3 · 따라하기: [Walkthrough](#)

데모 녹화 (asciinema)

데모	내용	링크
Self - Correction	위반 코드 → 하네스 블록 → 자동 수정 → 7/7 통과	Play
Build & Test	13개 테스트 통과 + 하네스 7/7 체크	Play
Live API	Ollama(qwen3:1.7b) 실시간 한국어 AI 응답	Play
Security	SpotBugs 정적 분석 + CycloneDX SBOM 생성	Play

모든 녹화: asciinema.org/~tedwon

Docs: github.com/.../docs

핵심 리스크 3가지

리스크	설명	대응
Skill Atrophy (기술 위축)	AI에 의존하면서 핵심 역량이 퇴화	설명할 수 없는 코드는 승인하지 마라
통제 없는 자율성	설계 없이 "만들어줘" → 엉뚱한 결과물	Playbook으로 방향 설정 먼저
맹목적 신뢰	AI는 자신감 있게 틀릴 수 있다	전문 지식으로 반드시 검증

AI는 전문가를 **대체**하는 것이 아니라 **증폭**하는 도구다.
"에이전트가 만든 코드를 설명할 수 없다면, 승인하지 마라."

프로젝트 구조

```
agentic-dev-playbook-with-harness/  
+-- CLAUDE.md / AGENTS.md / CHECKLIST.md ← Feedforward 규칙  
+-- .claude/  
|   +-- settings.json ← Hook 연결 설정  
|   +-- hooks/  
|       +-- pre-commit-harness.sh ← 7가지 검증 (Feedback)  
|       +-- protect-files.sh ← 파일 보호  
|       +-- post-edit-verify.sh ← 즉시 컴파일 확인  
+-- src/main/java/ ← Quarkus REST API  
+-- src/test/java/ ← 11개 테스트  
+-- docs/ ← ADR, Spec, Plan 저장소  
+-- demo/ ← 데모 녹화, 위반 예시 코드
```

Demo branch: github.com/tedwon/agentic-dev-playbook-with-harness/tree/feat/issue-2-quote-ai-chatbot

핵심 메시지

세 가지만 기억하세요

오늘의 핵심 정리

도구	역할
MCP AI Assistant	업무 도구 통합 — 다양한 서비스, 120+ 도구
Skills & Agents	전문 능력 확장 — 33 skills, 4 agents
Claude Code	개발 작업의 핵심 엔진 — CLI, IDE, Web
Harness Engineering	AI가 안전하게 자율 실행하는 구조
OpenClaw	언제 어디서든 AI 접근

1. HARNESS 설계가 모델 선택만큼 중요하다
2. 워크플로우로 시작하고, 필요한 곳에만 에이전트를 배치하라
3. 승인하는 코드를 반드시 이해해야 한다

오늘부터 시작하기

STEP 1: CLAUDE CODE 설치 + **CLAUDE.MD** 작성

에이전트에게 프로젝트 규칙, 코딩 컨벤션, 금지 사항을 알려주기

STEP 2: PRE-COMMIT HOOK 하나 추가

최소한 **컴파일 + 테스트** 자동 검증부터 시작

STEP 3: PLAYBOOK 4단계로 첫 번째 기능 개발

설계 → 실행 → 리뷰 → 검증 사이클을 한 번 돌려보기

GitHub: github.com/tedwon/agentic-dev-playbook-with-harness

Playbook, Hook 스크립트, 데모 코드 모두 공개

감사합니다

Q&A

Ted Jongseok Won

JBUG Korea

**"여러분의 소프트웨어 엔지니어링 전문성은 사라지는 것이 아니라,
AI와 결합해 더 강력해진다."**

"Humans steer. Agents execute."

AI-Driven Software Engineering

References

Harness Engineering · [Mitchell Hashimoto — My AI Adoption Journey](#) · [OpenAI — Harness Engineering](#) · [Anthropic — Harness Design for Long-Running Apps](#) · [Awesome Harness Engineering](#)

Agentic Development · [Anthropic — Building Effective Agents](#) · [Anthropic — 2026 Agentic Coding Trends](#) · [Karpathy CLAUDE.md Guidelines](#)

Papers · [Agentic AI Definition — arxiv 2603.27075](#) · [Agentic AI Survey — arxiv 2601.02749](#) · [Agentic as Marketing Term — arxiv 2506.01463](#)

Tools · [Claude Code](#) · [MCP](#) · [Ollama + Claude Code](#) · [Google Workspace CLI](#) · [NVIDIA OpenShell](#) · [Quarkus AI](#)

This Project · [GitHub Repo](#) · [Demo Branch](#) · [PR #3](#) · [Demo Recordings](#) · [Walkthrough](#)

Inspiration · [Julia Evans — Write a Brag Document](#) · [Pearce et al. — AI Code Security](#) · [Stack Overflow 2025 Survey](#)